MS3

Stato iniziale del sistema

Baseline architetturale prima dell’intervento del team 2025/26

Made with GAMMA

Executive Summary

MS3 — Medical Staff Shift Scheduler

Il problema centrale

L'obiettivo è **pianificare e gestire i turni** di una struttura ospedaliera (anche universitaria) **operativa 24/7**, con turni da 8 ore, garantendo il rispetto dei **vincoli normativi e organizzativi** (es. limiti di orario, presenza minima di medici strutturati, non ubiquità), valorizzando le diverse **categorie di personale** (strutturati, gettonisti, specializzandi) e bilanciando sostenibilità operativa, stress da festività, preferenze individuali su ferie/permessi e supportando le **richieste di scambio turno**, anche quando soggette ad approvazioni esterne

Il dominio comprende: medici, servizi, turni, assegnazioni concrete, preferenze/desiderata, festività, vincoli, richieste operative.

Interpretazione tecnica

Si intende **sostituire processi manuali**, costosi e soggetti a errore nella **pianificazione dei turni ospedalieri** con un **sistema informatico strutturato e tracciabile**, capace di formalizzare vincoli, preferenze e regole operative. L'obiettivo è rendere il processo decisionale **ripetibile, trasparente e verificabile**, riducendo i conflitti tra esigenze di copertura del servizio e preferenze individuali del personale, e migliorando complessivamente **l'efficienza organizzativa**.

La soluzione MS3

MS3 è un sistema web per la pianificazione dei turni del personale medico ospedaliero, che affronta il problema tramite un **motore di scheduling (**pianificazione complessiva su finestra temporale**) con regole esplicite** (constraint/vincoli hard/soft, log di violazioni, priorità tramite metrica interna di carico/disagio denominata **UFFA**/scocciatura) e strumenti di gestione delle eccezioni.

Made with GAMMA

Scopo del sistema e obiettivi operativi

Obiettivi funzionali osservabili

1

Generazione automatica

Generare automaticamente schedule a partire da dati e vincoli.

2

Rigenerazione

Rigenerare schedule in presenza di cambiamenti.

3

Gestione ciclo di vita

Gestire ciclo di vita di utenti, medici, servizi, turni, vincoli, preferenze e richieste.

4

Interfaccia web

Consentire consultazione/modifica operativa via interfaccia web.

Obiettivi non funzionali

Correttezza vincoli

Preservare i vincoli hard e rendere esplicite le violazioni residue.

Efficienza operativa

Ridurre il tempo planner necessario per ottenere una schedule valida.

Auditabilità

Mantenere tracce persistenti di decisioni, eccezioni e stato dei turni.

Made with GAMMA

Stakeholder e scenari d'uso

Planner / Pianificatore

Genera, valuta e stabilizza le pianificazioni.

Configuratore / Amministrazione

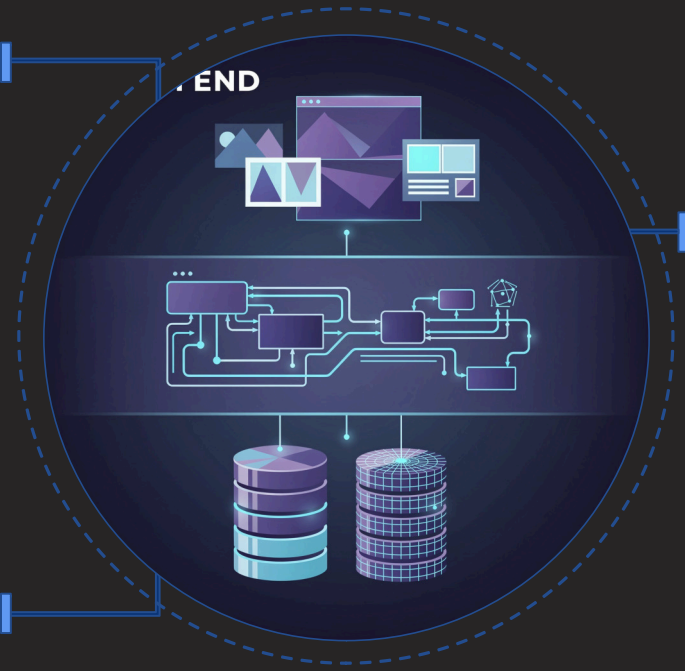
Governa regole, vincoli e anagrafiche.

Utente medico

Consulta turni, inoltra richieste di cambio turno e preferenze

Made with GAMMA

Architettura tecnica



Frontend
React SPA moderno

Backend
Java con Spring Boot

Database
PostgreSQL multi-tenant

L'architettura di MS3 si articola su tre livelli principali — backend, frontend e persistenza — con un modello multi-tenant che garantisce l'isolamento dei dati tra strutture organizzative distinte tramite partizionamento dei dati.

Made with GAMMA

Backend

Interpretazione tecnica

Il backend adotta una **separazione a layer controller-logica-accesso dati**, con forte specializzazione sul dominio scheduling e sui vincoli dichiarativi.

Stack tecnologico

Java 11

Spring Boot 2.1.x

Hibernate / JPA

Maven

Package principali

`rest`, `control`, `dao`, `entity`, `dto`, `config`, `exception`, `tenant`, `jpa_constraints`

Endpoint REST

Già ampiamente presenti nella baseline.

Dominio persistente — entità centrali

`Schedule`, `ConcreteShift`, `DoctorAssignment`, `Constraint*`, `Preference`, `Holiday`, `MedicalService`, `User`, `Waiver`, ecc.

Made with GAMMA

Frontend — React SPA

Interpretazione tecnica

La SPA implementa una **UI operativa orientata al workflow**, separando chiamate HTTP e logica di presentazione.

Struttura modulare

`frontend/src/API`, `views`, `components`, `layouts`, `utils`, `flux`

Comunicazione backend

Tramite moduli API dedicati: `ScheduleAPI.js`, `DoctorAPI.js`, `VincoliAPI.js`, ecc.

Viste principali nella baseline

- `ScheduleGeneratorView`
- `ScheduleView`
- `SingleScheduleView`
- `ShiftChangeView`
- `UserProfilesView`
- `ConfigurazioneVincoli`
- `MedicalServicesView`

Made with GAMMA

Database e multi-tenancy

Interpretazione tecnica

L'isolamento tenant avviene a **livello schema/logica applicativa**, soluzione coerente con requisiti di segregazione dati in contesti organizzativi multi-struttura.

DBMS

PostgreSQL come sistema di gestione del database.

Schema multi-tenant

Setup documentato con utenti/schema separati: `public`, `tenant_a`, `tenant_b`.

Bootstrap

Script di bootstrap per l'inizializzazione degli ambienti tenant.

Made with GAMMA

Configurazione e deploy

Interpretazione tecnica

La baseline è deployabile in ambienti development/integrazione, con **margini di hardening production** (segreti, standardizzazione pipeline, controllo configurazioni).

Esecuzione locale

Supportata tramite configurazione diretta dell'ambiente di sviluppo.

Containerizzazione

Supportata tramite `docker-compose` per ambienti integrati.

Configurazione

Guidata da **variabili ambiente** per flessibilità tra ambienti.

Made with GAMMA

Stato funzionale del sistema prima di quest'anno

Di seguito viene presentata una panoramica dello stato funzionale di MS3 **prima dell'anno accademico 2025-26**, articolata in tre aree: funzionalità implementate, aree parzialmente coperte o in evoluzione, e rischi/limiti tecnici della baseline.

Made with GAMMA

Implementato

Motore e flussi core di scheduling

Generazione e rigenerazione delle schedule pienamente operative.

Gestione completa delle principali entità dominio

Utenti, medici, turni, servizi, vincoli, preferenze, festività.

Workflow applicativi essenziali

Per planner e utenti, copertura dei flussi operativi principali.

Suite test Java significativa

Soprattutto su scheduler e vincoli, ma solo pre-multi-tenancy (`src/test/java/org/cswteams/ms3/...`).

Made with GAMMA

Parzialmente coperto o in evoluzione

Error handling / Observability

Standardizzazione trasversale di error handling/observability **non pienamente uniforme** su tutti i flussi.

Hardening security avanzato

Es. **MFA/TOTP** non ancora incluso nella baseline.

Made with GAMMA

Rischi e limiti tecnici della baseline

⚠️ Complessità dominio elevata

Molte regole interdipendenti richiedono **test robusti** per evitare regressioni.

⚠️ Debito operativo

Gestione segreti e policy deployment da **rafforzare in ambienti production**.

⚠️ Evoluzione funzionale

Introduzione successiva di moduli AI/sicurezza impone **compatibilità contrattuale e governance dati** più stringente.

Made with GAMMA

Qualità ingegneristica nella baseline

Testing

Fatti: ampia presenza di test su scheduler, vincoli e controller; presenti anche `src/test/README.md` e `src/test/TEST_RESULTS.md`.

Interpretazione: buona copertura sulle aree critiche di dominio; da completare la continuità end-to-end e la stabilità CI complessiva.

Error handling / logging

Fatti: package `exception` presente; gestione errori applicativi distribuita nei layer REST/control.

Interpretazione: approccio adeguato per validazione dominio, ma con spazio per una tassonomia errori più uniforme tra endpoint.

Sicurezza

Baseline funzionale ma **non ancora al livello massimo di hardening** richiesto da contesti ad alta criticità operativa.

Made with GAMMA